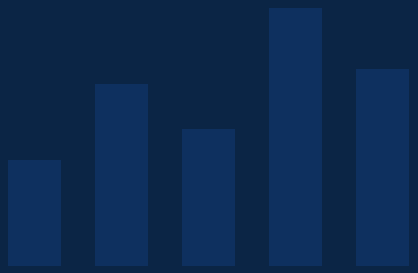
PROJECT PLAN

Agentic AI-Powered Data Analyst for Autonomous Business Intelligence

One-week implementation roadmap for an evidence-backed, adaptive analytics MVP

Plan, investigate, verify, explain.



DOCUMENT	Project plan and implementation proposal
SCOPE	One-week MVP: data upload, analytics tools, adaptive investigation, charts and report
TIMELINE	Seven-day intensive build plan for one student using coding agents
DATE	October 2026

Contents

A compressed implementation plan prioritizing a working end-to-end demonstration over production-scale infrastructure.

01 Responsibilities by layer . . . 3
02 Tools and technology stack . . . 3
03 Agent roles and end-to-end workflow . . . 4
04 Root-cause investigation design . . . 5
05 API design and data model . . . 6
06 Suggested repository structure . . . 7
07 Security, reliability, and responsible behavior . . . 7
08 Seven-day implementation roadmap . . . 8
09 Testing and evaluation plan . . . 8
10 Deployment and final deliverables . . . 9
11 Final recommendation and scope cuts . . . 9
A Appendix: Sample investigation output . . . 10

Project in one sentence

A conversational analytics system that plans and executes bounded multi-step investigations over uploaded data, adapts to findings, and presents verified, explainable business insights.

Defining features

01
Evidence-backed analytics
Every important number traces to a deterministic calculation.

02
Adaptive investigation
The next analytical step depends on earlier findings.

03
One-week scope discipline
A single orchestrator and small set of reliable tools.

How to use this plan

Build in vertical slices: each day should end with a working, testable capability. Coding agents can scaffold and implement modules, but the student remains responsible for reviewing changes, running tests, checking numerical correctness, and integrating the application.

01 Responsibilities by layer

Keep each layer small and explicit so that the MVP is easier to build, debug, and demonstrate.

Layer	Responsibility
Frontend	Collect questions, show dataset profile, progress, findings, and charts.
API	Validate uploads, expose analysis endpoints, create jobs, and return results.
Orchestrator	Use LangGraph to plan, call tools, evaluate results, and stop at bounded limits.
Analytics tools	Run deterministic profiling, aggregation, comparison, data-quality, and anomaly checks.
Verifier	Check calculations, references, reconciliation, and evidence behind claims.
Storage	For the MVP, keep uploaded files and analysis metadata simple; use local storage during development.

MVP success criteria

- A user can upload a CSV or Excel file and see a useful profile.
- A user can ask a business question in natural language.
- The system chooses from approved analytical tools rather than executing arbitrary generated code.
- The agent can run at least one follow-up step based on the initial result.
- The final answer includes supporting numbers, limitations, and at least one chart where appropriate.
- A small test suite confirms the key calculations against known answers.

Definition of done

The happy path works end to end on two or three sample datasets, and known-answer tests pass. Authentication, multi-tenant production hardening, cloud-scale queues, and advanced anomaly models are not required for this one-week target.

02 Tools and technology stack

Use familiar tools and avoid adding infrastructure that does not directly support the one-week demonstration.

Layer	Choice	One-week guidance
Frontend	React + TypeScript + Vite	Keep the current frontend approach; prioritize one responsive workspace.
UI and charts	Tailwind CSS + existing components; Plotly	Use the component library already in the project if one exists.
API	Python + FastAPI + Pydantic	Typed requests and response validation.
Orchestrator	LangGraph	One graph with clear state, conditional routing, and maximum step limits.
LLM	One tool-calling model + provider adapter	Use structured outputs; keep provider configuration in environment variables.

Layer	Choice	One-week guidance
Analytics	Pandas + NumPy	Core calculations, profiling, comparisons, and basic anomaly checks.
Query engine	Optional DuckDB	Add only if SQL over uploaded data is already straightforward or needed.
Metadata	Local files + simple job state; PostgreSQL optional	Do not let database setup delay the core analytics workflow.
Background jobs	Synchronous first or simple worker	Move to Celery + Redis only if blocking requests becomes a problem.
Testing	Pytest + HTTPX; basic frontend checks	Test calculation functions and API routes early.
Delivery	Docker optional; GitHub Actions if already configured	Deployment is a final-day stretch goal, not a blocker.

Defer until after the demo

- Kubernetes, Celery/Redis, OpenTelemetry, managed object storage, and multi-agent frameworks.
- Vector databases unless semantic search over documents is a real requirement.
- Multiple LLMs or a separate LLM for every agent role.
- Arbitrary LLM-generated Python execution inside the API process.
- Complex authentication, billing, team permissions, and advanced model evaluation dashboards.

03 Agent roles and end-to-end workflow

Represent the work as a bounded workflow, not a collection of loosely coordinated agents.

Component	Responsibility	Output
Profiler	Infer column types, missing values, duplicates, ranges, and useful metrics.	Profile + quality flags
Planner	Turn the user question into a structured plan using known columns and allowed tools.	Validated plan
Analytics tools	Compute aggregates, period comparisons, segment contributions, and quality checks.	Numeric results + provenance
Investigation controller	Decide whether findings justify another approved analysis or whether to stop.	Next-step decision
Verifier / report writer	Check totals and evidence, then explain results and limitations.	Verified answer + chart specs

End-to-end workflow

Step	Stage	Action
1	Upload	Validate file type, size, encoding, and parser behavior.
2	Profile	Show schema, sample rows, missing values, and candidate metrics.
3	Plan	Interpret metric, comparison period, dimensions, filters, and allowed tools.
4	Execute	Run deterministic calculations through approved functions only.
5	Evaluate	Check whether the first result answers the question; choose a bounded follow-up if needed.
6	Verify	Recalculate totals, percentages, and segment reconciliation; attach evidence references.
7	Present	Return answer, chart, assumptions, caveats, and investigation steps.

Bounded agent behavior

Set a maximum of 3–5 analytical tool calls per question, a small retry limit, and a clear stop condition. If a tool fails or evidence is insufficient, report the limitation instead of inventing a result.

04 Root-cause investigation design

Use a concrete business question to demonstrate adaptive analysis while avoiding unsupported causal claims.

Example question

“Revenue fell in September compared with August. Investigate the main contributors and explain what the data supports.”

Investigation sequence

Stage	What the agent does
A. Quantify	Compare total revenue across August and September; calculate absolute and percentage change.
B. Decompose	Group the change by region, product, or channel using available columns.
C. Prioritize	Rank segments by contribution to the overall change and inspect the largest movements.
D. Validate	Check missing values, duplicate records, inconsistent dates, and whether segment changes reconcile.
E. Follow up	If needed, inspect units sold, order counts, or average order value if those metrics exist.
F. Report	Summarize measured contributors, show a chart, state assumptions, and list data limitations.

Small approved tool set

Tool	Purpose
`profile_dataset`	Schema, nulls, duplicates, numeric summaries
`compare_periods`	Compare metric values between two periods
`group_metric_change`	Break down change by a categorical dimension
`test_data_quality`	Flag missing, duplicate, or inconsistent records
`detect_anomalies`	Apply a simple, documented statistical rule
`validate_reconciliation`	Check that subgroup totals align with overall totals
`create_chart`	Return a chart specification from verified results

Language rule

Say “the largest measured contributor was Region A” when supported. Do not say “Region A caused the decline” unless the data and analysis establish causation.

05 API design and data model

Keep the API minimal; implement only endpoints required by the core demonstration.

Method	Endpoint	Purpose
POST	`/datasets/upload`	Upload and validate a CSV/XLSX file; return dataset ID and profile.
GET	`/datasets`	List uploaded datasets for the current local session/user.
GET	`/datasets/{id}/profile`	Return schema, quality summary, and sample metadata.
POST	`/analysis`	Start an analysis question and return a job ID.
GET	`/analysis/{id}`	Return status, steps, findings, and final answer.
GET	`/analysis/{id}/events`	Optional SSE stream for live progress; polling is acceptable for MVP.
GET	`/analysis/{id}/report`	Return or download a simple report if export is completed.

Minimum metadata

Record	Minimum fields
datasets	id, filename, file_path, uploaded_at, schema/profile JSON
analysis_jobs	id, dataset_id, question, status, created_at, completed_at
analysis_steps	job_id, step_number, tool_name, input summary, output summary, status
findings	job_id, claim, metric, value, evidence reference, confidence/limitation
reports (optional)	job_id, report content or generated file path

Scope shortcut

For a one-week demo, SQLite or in-memory job state can be acceptable for local development. Preserve clear interfaces so PostgreSQL and durable background processing can be added later.

06 Suggested repository structure

A small structure helps coding agents work module by module and reduces accidental cross-file changes.

```
project/
  frontend/
    src/components/
    src/pages/
    src/api/
  backend/
    app/main.py
    app/api/datasets.py
    app/api/analysis.py
    app/orchestration/graph.py
    app/orchestration/state.py
    app/analytics/profiling.py
    app/analytics/comparisons.py
    app/analytics/data_quality.py
    app/verification/reconciliation.py
    app/reporting/
    tests/
      test_profiling.py
      test_comparisons.py
      test_workflow.py
  .env.example
  Dockerfile (optional)
  README.md
```

07 Security, reliability, and responsible behavior

Essential security and reliability checks

- Limit upload size and accept only explicitly supported file types; handle malformed files safely.
- Treat spreadsheet cell text as untrusted data, not as instructions to the LLM.
- Never execute arbitrary LLM-generated Python; call approved deterministic functions only.
- Limit rows, processing time, tool calls, retries, and model spend.
- Keep API keys in environment variables; commit only a .env.example file.
- Avoid logging raw sensitive rows; log job IDs, tool names, durations, and sanitized errors.
- If SQL is used, restrict it to read-only operations and validate table/column references.

One-week security boundary

This is a controlled demo, not a production multi-user service. Do not expose it publicly with sensitive uploads until authentication, isolation, file cleanup, and access controls have been reviewed.

08 Seven-day implementation roadmap

Target a working end-to-end path by Day 5, then use the final two days to test and stabilize.

Day	Focus	Tasks	Acceptance check
DAY 1	Foundation + ingestion	Set up project, environment, FastAPI routes, upload CSV/XLSX, validate files, profile schema and missing values.	Upload a file and inspect its profile.
DAY 2	Deterministic analytics	Implement metric aggregation, period comparison, group-by decomposition, data-quality checks, and unit tests.	Tools return correct results on sample data.
DAY 3	LLM + orchestration	Connect one tool-calling LLM; create LangGraph state, structured plan, tool routing, and bounded execution.	Ask a question and trigger the right tool.
DAY 4	Adaptive investigation	Add follow-up decision logic, reconciliation checks, evidence references, retry/stop rules, and uncertainty language.	Agent runs at least one useful follow-up step.
DAY 5	Frontend + charts	Build chat workspace, dataset profile view, analysis progress, findings, Plotly charts, and basic report output.	Full user flow works from upload to answer.
DAY 6	Tests + hardening	Test known-answer questions, fix integration bugs, handle invalid inputs/timeouts, and improve error messages.	Core tests pass; failures are explained.
DAY 7	Freeze + demo	Regression test, write README and setup steps, add Docker/deployment only if stable, prepare demo script and screenshots.	Reproducible demo and project explanation.

Daily execution rule

- Start each day by asking the coding agent to inspect the existing repository before editing.
- Request small changes with explicit files, interfaces, acceptance criteria, and tests.
- Run the app and tests yourself after each major change; commit a working checkpoint daily.
- If behind schedule, cut optional features—not validation, deterministic calculations, or end-to-end integration.

09 Testing and evaluation plan

A small, reliable demo is more valuable than a broad feature list with unverified results.

Evaluation area	One-week acceptance test
Numerical accuracy	Use 5–10 questions with manually calculated expected answers.
Tool selection	Check whether the agent selects the appropriate approved function.
Adaptive follow-up	Ensure a first result can trigger a relevant next analysis.
Evidence support	Every key claim includes a calculation/result reference.
Reconciliation	Subgroup contributions are checked against the overall change.
Failure behavior	Invalid file, missing column, unsupported question, timeout, and model error.

10 Deployment and final deliverables

Minimum final deliverables

- Working React frontend and FastAPI backend with a clear local setup guide.
- CSV/XLSX upload and data profile.
- A small library of deterministic analytics tools with tests.
- One bounded LangGraph workflow using a single tool-calling LLM.
- Adaptive root-cause investigation with evidence-backed findings and at least one chart.
- README with architecture, environment variables, limitations, sample questions, and screenshots.
- Demo dataset and a short scripted demonstration.

Deployment priority

Local execution is the baseline. Docker and cloud deployment are stretch goals only after the main flow passes tests. If a deployment is needed for evaluation, prefer the simplest service you already know rather than learning a new platform during the final day.

11 Final recommendation and scope cuts

Treat the week as an MVP sprint with a hard feature freeze.

> **Recommended MVP**
>
> React + FastAPI + Pandas + one tool-calling LLM + LangGraph + a small set of deterministic analytics tools. Focus on one compelling business investigation from upload through evidence-backed explanation.

Must keep

- Upload and profile a dataset.
- Perform accurate deterministic calculations.
- Use an LLM to plan and select approved tools.
- Run at least one adaptive follow-up analysis.
- Verify key values and communicate limitations.
- Show a chart and explain the evidence.

Postpone

- Production authentication and multi-user authorization unless required by the demo environment.
- Celery/Redis, object storage, Kubernetes, vector search, and advanced observability.
- Complex multi-agent collaboration, sophisticated causal inference, and advanced anomaly models.
- Large benchmark suites; start with 5–10 high-quality known-answer tests and expand after the deadline.

Appendix A — Sample investigation workspace

DATASET sales_sample.csv Rows: 12,450 · Columns: 9	**QUESTION** Why did revenue decrease in September?
FINDING 1 Revenue decreased by 8.4% month over month.	**FINDING 2** The largest measured contributor was the North region.
VERIFICATION Segment changes reconcile within tolerance.	**LIMITATIONS** Observed contribution does not prove causation.

Illustrative values only; replace them with results calculated from the uploaded dataset.

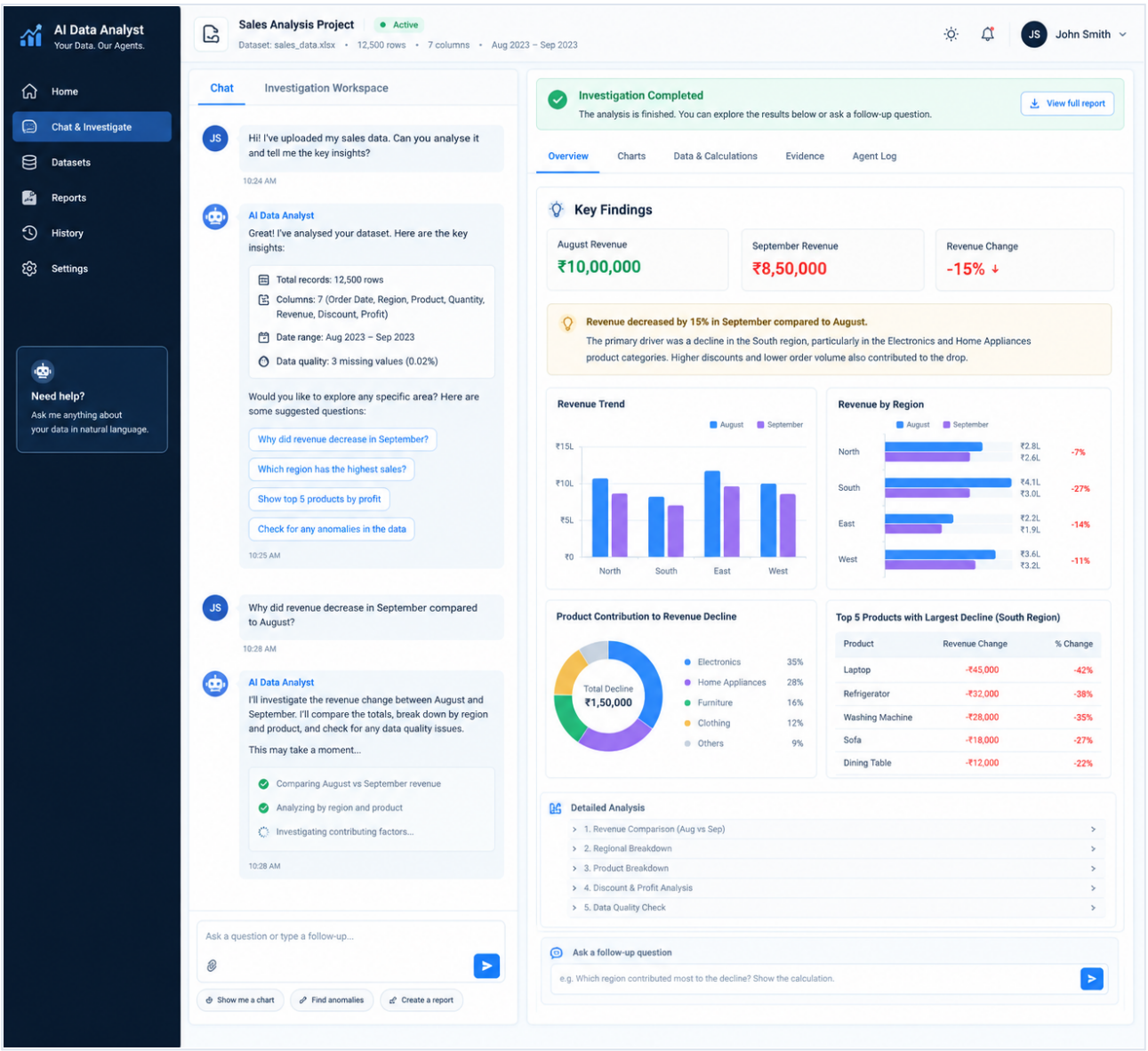


Figure A.1 *— Sample investigation workspace: chat, key findings, charts, and detailed analysis for the question "Why did revenue decrease in September?"*